Prosaic v2: standby agents, adversarial review, and a faster loop

Proposal and progress record. Branch `dev`, checkout `~/code/prosaic_dev`. W0 and W1 are built on `dev`; nothing is merged to `main`.

September 6, 2026

1. What thirty days of transcripts say

I parsed every Claude Code transcript on this machine from August 6 to September 6 (898 MB, 308 sessions, 82 subagent runs, 45 active days) and read the sync logs, launchd plists, settings and knowledge files of the live deployment.

Measure	Value
Interactive turns / active agent time	1,411 turns / 100 hours
Median turn, p90 turn (active seconds)	96 s / 633 s
Turns over 5 minutes	325 turns carrying 73 of the 100 hours
Tokens: cache read / cache write / output / thinking	6.4 B / 125 M / 16.4 M / 3.8 M
List-price equivalent	about $6,200 per month; 64 % of it is cache reads
Context size per API call (median, p90)	380 k / 800 k tokens; 47 % of calls above 400 k
Sessions spanning more than 36 hours	20 of 52, carrying 79 % of all calls
Effort level	`high` on 31,512 calls, `medium` on 101
Turns that delegated to a subagent	47 of 1,411 (3 %), mostly to Fable or Opus
Routine prompts (build, open, commit, push)	53 turns, median 39 s, p75 146 s
Scheduled triage runs	34, median 3.9 min, about 20 calls each, on the 1M Fable model with no model, turn or budget cap

The slowness is context, not model. Every call re-reads a 400k to 800k token prefix. Prefill on that much cache is seconds per call before any thinking starts, and a routine three-call build turn pays it three times. The long-lived sessions are the cause: twenty sessions that ran for days carry four fifths of all traffic. Switching Fable for Sonnet would not fix this. Starting sessions fresh, keeping instructions small, and pushing bulk reading into subagents would.

Routine commands are cheap when they stay routine. A bare “rebuild” takes 39 seconds at the median. The long tail is the model quietly fixing a source file when the build warns, which is real work wearing a routine prompt. The fix is a command path that runs the CLI and stops, so fix work becomes a separate, visible decision.

Effort is never tuned. Nearly every call ran at `high`. Thinking was 23 % of all output. There is no per-task effort or model routing anywhere in prosaic or in the harness settings.

The automation that exists is half-blind. Twice-daily sync and triage run and work, but connector failures are logged as `ok` because a `sed` in the pipeline masks the exit status. One connector has been broken for three weeks without a signal. Triage runs the most expensive model with permissions bypassed and no cap.

Knowledge has outgrown its rule. The largest matter’s KNOWLEDGE.md is 3,556 lines, about 82k tokens, with fourteen date-titled sections appended as a diary, two “as of August 13” snapshots superseded by later entries, zero cross-links and no tooling to notice any of it. TODO.md is 1,555 lines for 41 items.

2. Audit verdict on prosaic

The core bet holds: model drafts Markdown, deterministic code renders it, one seam (`cli/agent-run`) reaches whatever headless agent is installed, flows (`flows/run.py`) chain agent, command, judge and gate steps with a human approval file. That is the right skeleton for everything below. The gaps:

- **No model or role selection.** `agent-run` picks a CLI, never a model. Flows have no per-step provider. The judge, triage, and drafting all run on the one default.
- **The adversarial pass exists as a seed and is opt-in.** `flows/draft-review.yaml` already does opposing-counsel review, revise, judge, gate. Nothing runs it before `--final` or `sc sign`, there is no cite check, no clerk or bench persona, and no data source for how opposing counsel actually behaves.
- **No hooks, no slash commands, no shipped user skills.** The only agent-facing surface is prose SKILL.md files, two of which fail the repo's own 120-line test, so the suite is red.
- **Deployment is ahead of upstream.** The private deployment carries ADR-0039, `sc build-doc`, the build manifest, read-coverage triage and revised skills that prosaic lacks. The prosaic-first rule is currently inverted.
- **Push is blocked.** The pre-push leak guard rejects three files and one commit message on `main` for a form-number pattern, so eighteen local commits and the new `dev` branch cannot reach GitHub until scrubbed.
- **Dead and stale.** `prosaic/` holds only `.pyc` leftovers; `sc clean` judges by config, not age, so 70 files older than 30 days in `out/` are invisible to it.

3. Proposal, ranked by payoff per week of work

W0. Unblock and realign (small, first). Scrub the form-number fingerprints, push `main` and `dev`. Port the deployment-only changes up into prosaic so the deployment is a pure downstream again. Fix the two over-long skills, delete the dead package directory. Nothing else should land until `dev` pushes clean.

W1. Fast path for routine work (small, largest felt-speed win). Ship `.claude/commands/` from the matter template: `/build`, `/rebuild`, `/open`, `/commit`, `/clean`, `/status`. Each runs the CLI, prints stderr warnings verbatim, and stops; a warning that needs a source edit is reported, not fixed. Route them to a cheap model. Add a `SessionStart` hook that injects a short matter brief (parties, posture, next dates, top TODO items) instead of relying on the model to read the whole knowledge file. Adopt one session per task. Expected: cache-read spend and per-call latency roughly halved, and "rebuild" becomes a ten-second command.

W2. Standby loops (medium). A `prosaic-supervisor` per matter, installed by `sc schedule`, replacing the two 12-hourly plists:

- *Inbox watcher.* launchd `WatchPaths` on `inbox/` and each connector's staging directory; triage starts within a minute of a file landing, on Fable, with a turn cap and budget set per role.
- *Honest sync.* Fix the exit-status masking, write a run summary to `.state/`, surface failures in the daily brief.
- *Nightly knowledge refinement.* A flow that reads KNOWLEDGE, INDEX, MANIFEST and the day's commits, proposes integrated edits and a list of open questions, and stops at a gate. It never writes KNOWLEDGE unattended.
- *Retention.* `sc clean --older-than` for `out/` renders and `.flow/` run directories, still report-first, never touching `assets/`, `pleadings/` or `processed_files/`.
- *Daily standup.* A `/standup` command that opens a 30-minute interactive session over the refinement's questions plus QUESTIONS.md, writes answers into the right knowledge files, and commits as `record`. The nightly flow queues, you answer once a day.

W3. Pre-signature adversarial gate (medium). A `pre-signature.yaml` flow that `--final` builds and `sc sign` require, keyed to the source hash:

1. *Cite check* (deterministic first): extract every statute, rule and case citation; verify form and existence against a local authority cache, flag anything unverified for a human.
2. *Clerk persona:* filing compliance, form boxes, service, page limits, caption and exhibit rules.
3. *Skeptical bench persona:* what the judge will not believe, what is unsupported, what is asked for without authority.

4. *Opposing counsel persona*, briefed from an `oppo_profile.md` the refinement loop maintains from observed filings and correspondence: how they have attacked before, what they will move to strike, what they will say you omitted.
5. Human gate with the four reports side by side.

Each step declares its own provider and effort so personas can run on different models. Until W5 exists, "different model" means a different Claude model or a local model, not another vendor (see below).

W4. Knowledge as a directory (medium). An ADR replacing the single file with `knowledge/` topic and entity files, each with front matter (`aliases:`, `keywords:`, `entities:`, `related:`, `sources:`, `updated:`) that doubles as a link graph and a grep surface, and KNOWLEDGE.md reduced to an index with one line per topic. A linter, `sc knowledge check`, enforces absolute dates, no date-titled sections, every file indexed, every link resolving, and flags topics untouched since a later docket event. Migration of the live matters is a gated agent task you approve file by file. The nightly refinement loop and the session brief both become cheap once knowledge is topical.

W4a. Guaranteed extraction and recall (medium, part of W4, raised to critical). A real failure motivates this: asked whether a particular lawyer interaction appeared in a Bates-stamped production, the agent searched several times, reported that it did not, and was wrong; the passage was prominent. A false negative about the record is the worst failure this product can have. So the knowledge layer gets three mechanisms:

1. *Per-document extraction at triage.* Every triaged document gets a structured sidecar (people and entities, dates, events, statements and admissions, each with a page or Bates cite), written by Fable under the read-every-page discipline, alongside the prose INDEX row.
2. *Knowledge as a referential graph of Markdown files.* Each topic and entity file carries YAML front matter (`aliases:`, `keywords:`, `entities:`, `related:`, `sources:` with page or Bates cites, `updated:`) and a body written deliberately keyword-dense: name variants, roles, OCR misspellings seen in the record, dates in more than one format. The front-matter links form the graph; the keywords make the whole tree greppable at ripgrep speed. A derived entity index is rebuilt from the front matter and is never the place a fact lives.
3. *Three-tier search, cheap and exhaustive.* `/find` runs in order: the explicit graph (front-matter links and the entity index), then ripgrep across `knowledge/`, every sidecar and every text layer, then a Sonnet-class agent that reads the hit set, follows `related:` links to two hops, and answers with cites or reports the full searched scope. Fable is spent on writing the files at triage, not on reading them back. "Not in the record" is only sayable after all three tiers ran, with the scope shown; anything image-only without a sidecar is reported as unsearched, not absent.

W4, built September 7 (ADR-0043). `knowledge/` is a vault of typed notes with front matter as graph and grep surface; `KNOWLEDGE.md` is the generated index; `sc knowledge init|new|check|index|migrate` and a linter enforce the rules; the vault is Obsidian-compatible and Obsidian-optional, and opening it in Obsidian is the acceptance check. `sc upgrade <matter>` runs every deterministic migration in order and the `migrate-knowledge` skill documents the agent-and-human fold of a legacy single file into notes. The largest matter is the canonical upgrade target.

W4a, first slice, done September 6 (ADR-0041). Coverage is now a computed property: `sc text audit` classifies every PDF, image, DOCX and audio file; `sc text ensure` OCR-supplements and writes page-marked sidecars with a provenance header, never touching an original or a human sidecar; `sc find` lists everything not provably covered as unsearched and exits 2 while any such document exists; the sync runs `ensure` before the agent; the brief shows the coverage line. On the largest matter the audit found 1,039 PDFs with 97 sidecars, so the first `ensure` there is a long batch and a decision for you (it writes about 900 sidecars beside the originals).

Architectural default, decided September 6 (ADR-0042): derived artifacts live in parallel directories, never beside originals. `derived/text/<path>.txt` and `derived/ocr/<path>` mirror the matter; `out/` and `.state/` already worked this way. Source directories hold received material and human-authored files only, which also settles the `pleadings/` question. Legacy siblings are read and `sc text migrate` moves the tool's own into the tree. Future derived kinds (`derived/extract/`, `derived/handwriting/`) follow the same rule.

W4b. Handwriting review interface (medium, requested September 6). Machine OCR of handwriting is poor, and a wrong guess in a sidecar is a false negative waiting to happen. A local-only web page (privileged material never leaves the machine) lists cropped rectangles of handwritten or low-confidence regions, each with the machine's current guess and a correction field; submitting a correction writes it into the document's sidecar as human-verified text with page and box, and clears the item. Detection starts from tesseract word confidences,

with a handwriting model later. Depends on the W4a sidecars.

W5. Backend choice and the privilege boundary (large, staged).

- *Now:* an `agent:` block in deployment config and `matter.yaml` naming provider, model and endpoint per role (triage, judge, clerk, bench, oppo, drafting). `agent-run --role` reads it. Document the self-hosted path: Claude Code or Codex CLI pointed at an OpenAI-compatible endpoint serving Qwen, GLM, Kimi or gpt-oss. Every role defaults to local or first-party; nothing goes to a second vendor by default.
- *Later:* a privilege column in INDEX.md set at intake (none, attorney-client, work product, psychotherapist-patient, medical, other); an entity map built from `matter.yaml` and INDEX; a `sanitize` flow step that produces a pseudonymized working copy in the run directory; a hard runner rule that a step whose provider is marked `external` receives only sanitized inputs; and an append-only log of what left the machine and how it was transformed.

A caution on W5. Pseudonymization reduces exposure; it does not by itself preserve privilege, and whether a given disclosure waives anything is a legal judgment, not a tooling property. The system's job is to make the boundary explicit, default to local, and leave an audit trail. I would not send drafts or privileged records to a second vendor, even for the opposing-counsel pass, until the sanitizer exists and you have decided the legal question.

3a. What it should save: an estimate

These are list-price equivalents computed from the same month of transcripts, holding the work constant and changing only how it is run. They are estimates; the mechanisms are measured, the mix is assumed.

Lever	Assumption	Saving per month
Fresh sessions, session brief, subagents for bulk reading (W1, W2)	Context per call capped near 150k instead of a 380k median	about $2,500 (41 %)
Reviewer-directed small changes on Opus, Fable for drafts (decision 2)	Half of current Fable calls move to Opus	about $900 (15 %)
Cheap models for routine commands; turn and budget caps on triage, which stays on Fable (W1, W2)	Sonnet or Haiku for build, open, commit, clean	about $100 (2 %)
Combined	Levers overlap, so less than the sum	about $3,000 to $3,300 (50 %), from $6,150 to roughly $3,000

Thinking tokens are not a cost lever: all 3.8 M of them cost about $125. Effort tuning is worth doing for latency, not for money.

Speedup. Measured on this month's calls, median API latency rises with context: 5.6 s per call under 100k tokens, 9.5 s at 200k to 400k, 13.2 s above 600k. Prefill-dominated calls (under 400 output tokens) go from 3.8 s to 7.7 s across the same range. At the same context, Opus answers in roughly 70 % of Fable's time. So:

- A typical drafting or research turn, about nine calls today, should run 35 % to 45 % faster from context alone, and about 2x faster where Opus takes the small directed changes.
- A routine command (build, open, commit, clean) goes from a 39-second median to a few seconds, because the deterministic path makes no model call, or one short call on a cheap model. Call it 5x to 10x on those turns, which are about 40 % of all prompts by count.
- Scheduled triage stays on Fable and keeps its 4 to 5 minutes per run; the gain there is latency to start, within a minute of a file landing instead of up to twelve hours later.
- Overall, across the month's mix, roughly half the wall-clock time per unit of work, with the gains concentrated exactly where the slowness is most felt.

Standing constraint (added September 6, 2026). The deployment's practice-area form adapters, the descriptors, blanks and specs in the private module mounted under `modules/`, and the overlay engine that drives them, must keep working through every workstream. Any change to the form engine, descriptor schema, registry, `sc form` or module discovery either fills the existing module descriptors unchanged, verified against the deployment before a merge to `main`, or ships a mechanical migration script and doc with the change. The descriptor contract is treated as a public API. Concretely: the module today holds eleven descriptors and one test, so the gate is a new `sc form check --all` that discovers every descriptor across built-in, `modules/` and `local/` layers, fills each with fixture data, flattens, and renders the geometry preview, run in the deployment against the candidate engine before any merge to `main`. Building that check is part of W0.

4. Sequence and decisions (resolved September 6, 2026)

Order: W0, W1, W2 (watcher and honest sync first), W3, W4, W5-now, then W5-later. W1 and W2 are a week or two together; W3 and W4 a week or two each; W5-later is open-ended.

1. `main` is production; `dev` is the working branch, checked out at `~/code/prosaic_dev`. Decided.
2. Fable stays the drafting default for initial drafts and major changes. Small, directed changes that a reviewing frontier model specifies are implemented on Opus. Routine commands go to a cheap model. **Triage and first-pass integration of new documents stay on Fable**: they are heavyweight, critical work, not clerical work (revised later the same day, see the recall requirement below). Decided.
3. Knowledge-directory ADR direction approved; migration remains gated file by file. Decided.
4. For the owner's own matters, every role may run on any model and any vendor now; no sanitizer gate applies. The privilege boundary in W5-later is built for deployments serving other people, not as a precondition here. Decided.
5. `/standup` runs at 9 am daily by default. Decided.

5. Progress (as of September 6, 2026, evening)

W0, done and pushed. The leak-guard hits were scrubbed and the unpushed history reworded, so `main` and `dev` both push. The deployment's changes are ported upstream (ADR-0039, `sc build-doc`, the build manifest, read-coverage triage, revised skills), the two over-long skills are demoted under the cap, and the suite is green. `sc form check` exists as the descriptor gate: run against the live deployment with the candidate engine it passed all 27 forms (11 module, 4 local, 12 built-in) and surfaced one pre-existing preview crash on a local descriptor whose `yes`/`no` keys parse as YAML booleans; the engine now tolerates that and the check names it.

W1, done and pushed. ADR-0040 records the decision. Four CLI commands: `sc brief` (the one-page session orientation), `sc open` (an envelope's built PDFs), `sc find` (ripgrep over every text file and sidecar, with every PDF lacking a text sidecar listed as unsearched), and `sc harness install` (writes the bundle with the checkout path resolved; `sc init` does the same). The bundle in `templates/matter/.claude` carries a SessionStart hook that runs the brief and seven slash commands: `/build`, `/build-doc`, `/open`, `/clean`, `/status` relay CLI output on Haiku 4.5 at low effort and never edit a source; `/commit` and `/find` run on Sonnet 5 at medium effort. `docs/harness.md` and eleven tests cover it. Full suite: 690 tests, green.

Deployed September 7. `dev` is merged to `main` and into the deployment; every git-backed matter has the bundle; the scheduled sync runs text coverage. The largest matter is on every convention: text coverage, derived tree, vault with 191 notes (staging kept for review). Tests of the `/find` procedure with a Sonnet reader on two live matters answered correctly with cites; the first exposed that the missing Bates page lived in the related matter, which `related_matters` now covers, and the second exposed filename-only triage, which the checklist gate now makes explicit. W2, W3, W4b and W5 remain.

Workstream	State	Commits on `dev`
W0 unblock and realign	done	leak scrub on `main`; 6 port commits; `sc form check`

Workstream	State	Commits on `dev`
W1 fast path	done	ADR-0040; `sc brief/open/find/harness`; bundle; docs
W2 standby supervisor	next	
W3 pre-signature gate	planned	
W4 knowledge vault	built; the largest matter folded into 191 notes, staging awaits review	ADR-0043; `sc knowledge`; `sc upgrade`; migrate-knowledge skill
W4a recall	first slice built	ADR-0041; `sc text`; `sc find` with summary, `--all`, related matters, checklist gate
W5 backend choice, privilege boundary	planned	

Appendix: decision records made under this proposal

The ADRs accepted while W0, W1, W4 and the first W4a slice were built, reproduced from `design/adr/`.

ADR-0040: Routine operations are harness commands, not deliberation

Status: Accepted (September 6, 2026)

Context

A month of session transcripts from a live deployment showed where the time goes. Every routine request — build this envelope, open the PDFs, commit, report stale output — was a prose instruction to a frontier model working inside a session whose context had grown to several hundred thousand tokens. The model re-read the skill, decided how to run the CLI, ran it, read the result, and decided what to say. A bare rebuild took forty seconds at the median; when the build warned, the model fixed the source on its own initiative and the turn ran for minutes. Median context per call was 380k tokens, and per-call latency roughly doubled between a 100k and a 600k context. The cost was not the model; it was re-reading a huge prefix to do something a shell already knew how to do.

The system already has the deterministic surface: `sc build`, `sc build-doc`, `sc clean`, `sc list`, `sc form`. What it lacked was a way for the harness to reach that surface without a round of thought, and a way to start a session with the two-minute orientation instead of the whole knowledge file.

ADR-0020 made `cli/agent-run` the one seam to a headless agent and asked that harness-specific files be labeled as such. ADR-0021 and ADR-0029 made skills the home of agent-facing procedure. ADR-0039 made `sc` the only build interface.

Decision

1. **Routine operations ship as harness commands** in the matter template, under `templates/matter/.claude/skills/`: `build`, `build-doc`, `open`, `clean`, `status`, `commit`, `find`. Each runs the CLI through the harness's inline-execution syntax before the model sees anything, then relays the result. The model's job is to read output, not to decide how to obtain it.
2. **A relay command runs on the cheapest adequate model at low effort.** Build, open, clean and status declare a small model; commit and find declare a mid model because they write a message or read hits. Drafting, triage and integration are untouched by this ADR and stay on the strongest model available.
3. **A relay command never edits a source.** A build warning that needs a source edit is reported with file and line and stopped on; the edit is a separate, visible decision. This is what keeps "rebuild" from silently becoming "rewrite".
4. **Sessions start from a brief, not from the knowledge file.** A `SessionStart` hook prints `sc brief`, a deterministic page: case, envelopes, dates from the knowledge file's calendar headings, the top of TODO,

recent docket commits, sync and inbox state, and the routine commands. The knowledge file is read by section, on demand.

5. **The bundle is prosaic's, installed by `sc init` and refreshed by `sc harness install`.** The checkout path is resolved at install time from a placeholder, because these files run shell commands. A matter customizes through `.claude/settings.local.json` and skills under other names; the bundle's files are overwritten on refresh.
6. **The bundle is harness-specific and says so.** It targets the Claude Code skill and hook format. Another harness gets its own bundle directory; the deterministic commands they wrap (`sc brief`, `sc open`, `sc find`) are harness-neutral and live in `cli/sc`.

Consequences

- A routine turn costs one short call on a small model with a small context, or none. The measured tail — minutes spent fixing sources under a "rebuild" prompt — becomes a request the user sees and approves.
- `sc brief` gives every session the same orientation; a session that needs more reads the section it needs. Loading a whole knowledge file into context is now a choice, not the default.
- `sc find` is the first tier of record search: ripgrep over every text file and sidecar, and an explicit list of PDFs that have no text layer and were therefore not searched. "Not in the record" is only sayable after that list is empty or accounted for. The knowledge graph and entity index that make the later tiers cheap are a separate decision.
- The per-command `model:` values are a starting point, chosen by the measured task shape, and are expected to be tuned per deployment.
- Nothing here changes `agent-run` or the flow runner; batch agent work keeps its one seam.

ADR-0041: Searchability is audited and repaired, never assumed

Status: Accepted (September 6, 2026)

Context

The record search added under ADR-0040 (`sc find`) greps text files. A PDF is searchable only through a text sidecar, and the first version counted a PDF as searched when an `_ocr.pdf` sibling existed — which ripgrep cannot read. Measured on the largest live matter: 1,039 PDFs, 97 with a `.txt` sidecar, 31 with an `_ocr` sibling and no sidecar, 911 with neither. A search over that record could see under a tenth of it and still say "not found".

The triage conventions asked the agent to write sidecars and to OCR scans. A convention an agent must remember is not a guarantee; the sync log shows connector failures logged as `ok` for weeks for the same reason (a pipe masking an exit status). Nothing checked coverage, so nothing noticed.

Decision

1. **Coverage is a computed property of the matter, not a convention.** `sc text audit` classifies every PDF, image, DOCX and audio file under the matter (outside `out/`, `.state/`, `.git/`, `.flow/`) as `searchable`, `unverified-sidecar`, `needs-sidecar`, `stale-sidecar`, `needs-ocr`, `transcript-needed`, `unsupported`, `unreadable`, or `untriaged` (inbox). A PDF is `searchable` only when a current sidecar written by the tool covers every page; an `_ocr` sibling by itself proves nothing.
2. **Repair is mechanical.** `sc text ensure` OCR-supplements PDFs whose pages lack a usable text layer (`ocrmypdf --skip-text`, or `--force-ocr --pages` for pages whose text is useless: image-bodied, garbled), then writes a `.txt` sidecar from the best source with a provenance header and `[[[ page k of N ]]]` markers so a hit carries a page cite. Images get `<stem>.ocr.txt` through tesseract; DOCX gets `<stem>.txt` through pandoc. Audio is reported only; the local transcription pipeline is a separate, human-supervised step.
3. **Originals and human sidecars are never touched.** Everything the tool writes is a sibling. A `.txt` without the tool's header is someone's transcription: it is searched, reported as unverified, and never overwritten. The tool rewrites only its own sidecars, and only when the document or its `_ocr` sibling is newer.

4. **Search refuses to hide a gap.** `sc find` takes its UNSEARCHED list from the audit, prints the coverage line, and exits 2 whenever anything triaged is not searchable — with or without hits. `--ensure` runs the repair first. The `/find` command uses `--ensure`.
5. **The sync runs ensure.** After the connectors and before the agent, `matter_sync.sh` runs `sc text ensure`, so a document that arrives is searchable before anyone reads it. The same change makes the connector loop test node's exit status rather than sed's.
6. **The brief carries the coverage line.** Every session starts by seeing how much of the record is searchable.
7. **Classification is cached** in `.state/text_coverage.json` keyed by path, size and mtime (of the document, its `_ocr` sibling and its sidecar), so an audit of a thousand documents is fast and a changed file is always re-surveyed.

Consequences

- Machine sidecars double the file count beside born-digital PDFs (every gmail export gains a `.txt`). That is the price of a record ripgrep can search in full; the header makes machine text distinguishable from a human transcription at a glance.
- "Not in the record" acquires a checkable meaning: the coverage line says how much was searchable and the UNSEARCHED list names the rest.
- The first `ensure` on a live matter is a long batch (hundreds of text dumps, OCR for the scans); later runs touch only new or changed files.
- Handwriting remains a gap tesseract fills poorly. A review interface for correcting handwritten regions, whose corrections become human-verified sidecar text, is the planned next step (W4b in the v2 proposal).
- The triage prompt no longer asks the agent to write or skip sidecars; it tells the agent they exist and to check the audit.

ADR-0042: Derived artifacts live in parallel directories, not beside originals

Status: Accepted (September 6, 2026)

Context

Every derived artifact the system produced for a document was placed beside it: `<stem>_ocr.pdf`, `<stem>.txt`, `<stem>.srt`, `<stem>.ocr.txt`. ADR-0041's coverage pass made the cost visible at scale. On the largest live matter it wrote 1,067 files into evidence directories in one run, and 58 OCR'd copies of court-filed documents landed in `pleadings/`, which the workspace contract reserves for the court's copy and nothing else — precisely because a derived PDF is indistinguishable from a filed one in a listing. Sidecars also complicate every directory operation: an INDEX row per "non-sidecar file", `.gitignore` patterns that must name each sidecar kind, triage prompts that must explain what to skip, and a doubled file count in every folder a human browses.

`out/` (build products) and `.state/` (connector state) already follow the other pattern: a parallel tree, one place, regenerable.

Decision

1. **Derived artifacts live under `derived/<kind>/`, mirroring the matter's own layout.** For a document at `assets/gmail/x.pdf`: `derived/text/assets/gmail/x.pdf.txt` holds the page-marked text; `derived/ocr/assets/gmail/x.pdf` holds the OCR'd copy. The full filename is kept and a suffix appended, so `x.pdf` and `x.png` never collide. Future kinds follow the same rule: `derived/extract/` for structured per-document extraction, `derived/handwriting/` for review crops.
2. **Source directories hold received material and human-authored files only.** `assets/`, `pleadings/`, `discovery/`, `processed_files/` and the rest contain originals, INDEX and MANIFEST files, and human notes. Nothing a tool regenerates goes there.
3. **Tracked or ignored by kind.** `derived/text/` is tracked: small, and it is what the record is searched through. `derived/ocr/` is ignored: large and regenerable by `sc text ensure`. The matter template's `.gitignore` says so; `sc text migrate` adds the line to an existing matter.
4. **Legacy siblings are recognized, read, and never silently abandoned.** The audit finds a `_ocr.pdf` or `.txt` beside an original, uses it, and notes the legacy location. `sc text migrate` moves what the tool wrote (its

header marks them, and its `ocr:` line says whether it made the OCR copy) into `derived/`, with `git mv` for tracked files. A `.txt` beside an original with no header is a human transcription: searched, reported as unverified, left in place.
5. **Human transcriptions may also go under `derived/text/`.** The default for new work is the parallel tree regardless of author; the header, or its absence, says who wrote it.
6. **`sc find` cites the original.** A hit inside `derived/text/...` is reported as the original document's path and page, with the text file named second, so a hit reads as a cite rather than as a path into a cache.

Consequences

- `pleadings/` is again the court's copy and nothing else; the 58 derived copies move out under migration.
- Evidence directories stop doubling in size; a browser of `assets/` sees documents, not documents and their shadows.
- One `.gitignore` line covers every OCR copy; one directory holds every text file; the coverage audit has one place to look before falling back to legacy locations.
- The INDEX, layout docs, triage prompt and workspace contract lose the sidecar vocabulary. INDEX rows describe documents; the derived tree needs no rows.
- Matters created before this ADR carry legacy siblings until migrated; `sc text audit` names them and migration is one command. Agent-made `_ocr.pdf` siblings that predate the tool move only with `--include-legacy-ocr`, because INDEX rows may name them.
- Deletion of an original leaves an orphan under `derived/`; `sc clean` should learn to report derived files whose original is gone (not done here).

ADR-0043: Knowledge is a vault of notes, not one file

Status: Accepted (September 7, 2026)

Context

The matter template gave durable knowledge one file, `KNOWLEDGE.md`, with a rule: integrate new facts into sections, never append a log. On the largest live matter the file reached 3,556 lines, about 82,000 tokens, with fourteen date-titled sections appended as a diary, two "as of" snapshots superseded by later entries, and no cross-links. Every session that consulted it paid for the whole file on every call, and the model doing the consulting was the most expensive one available. When asked whether a particular interaction was in the record, an agent read the file several times and answered wrongly.

The rule failed because nothing enforced it and because one file gives a fact no address. A person, an event and an issue all live in the same scroll; the only way to find one is to read, and the only reader that can is a model.

Decision

1. **Knowledge is a directory of notes**, `knowledge/`, one note per person, organization, event, topic, issue or filing, in a folder per type. `KNOWLEDGE.md` at the matter root becomes the index, regenerated between two markers by `sc knowledge index`.
2. **Front matter is the graph and the grep surface.** Every note carries `title`, `type`, `aliases` (every name variant the record uses, OCR misspellings included), `tags`, `related` (wikilinks), `sources` (path, pages, note), `updated` (an absolute date) and `verified` (machine or human). Events carry `date`; issues carry `status`. Bodies are keyword-dense prose with absolute dates and no dated headings.
3. **The vault is Obsidian-compatible and Obsidian is optional.** Wikilinks resolve by filename stem; `aliases` and `tags` are Obsidian's own keys; the matter directory opens as a vault with backlinks and a graph for a human reader, and `.obsidian/` is ignored. Nothing an agent or a tool does depends on Obsidian, and no fact may live only in a view that renders inside it. Opening the vault in Obsidian and finding useful backlinks is an acceptance check, not a dependency.
4. **A linter enforces the rules.** `sc knowledge check` fails on a missing title, type or absolute `updated` date, a dated heading, an unresolved link, a source path that does not exist, an alias claimed by two notes, or a note absent from the index; it warns on orphans, relative dates, and staging notes.

5. **Search consults the vault first.** `sc knowledge index` also writes `derived/knowledge/entities.json`, an alias index; `sc find` prints matching notes before the ripgrep pass, and `sc brief` draws upcoming events and open issues from event and issue notes.
6. **Migration is staged and never destructive.** `sc knowledge migrate` copies a single-file KNOWLEDGE.md whole into `knowledge/_migration/KNOWLEDGE.legacy.md`, splits it by H2 into staging notes with front matter, and turns KNOWLEDGE.md into the index. The `migrate-knowledge` skill then has an agent fold staging into real notes, gated by a human reading the result; staging is a warning until it is gone and a human removes it.
7. **Fable writes, cheap models read.** Extraction and integration of new documents into notes is Fable-class work at write time. Reading back is ripgrep, the alias index, and a Sonnet-class reader following links; the vault is what makes that cheap.

Consequences

- A fact has an address: a note, a heading, a source with a page. Recall becomes a lookup, and "everything about X" is the backlinks of X.
- Sessions load the brief and the notes they need, not the whole history. The cost profile of every routine session drops.
- The triage prompt and the workspace contract stop saying "update KNOWLEDGE.md" and say "create or update the note"; the `record` commit type is unchanged.
- Existing matters carry a staging directory until an agent and a human finish the migration; `sc upgrade` starts it and the skill documents it. The legacy file is never deleted by a tool.
- Human editing in Obsidian is the cheapest review there is; the daily standup can happen there. Corrections made in Obsidian are ordinary file edits and commit as `record`.
- Two notes about the same person under different names are the new failure mode; the alias check and the migration skill's merge step exist for it.